

Table of Contents

LIST OF ABBREVIATIONS	iii
LIST OF FIGURES	iv
LIST OF TABLES	v
1. Introduction.....	1
2. Problem statement.....	2
3. Objectives	2
4. Methodology	2
4.1. Requirement Identification	2
4.1.1. Study of Existing System / Literature Review.....	2
4.1.2. Requirement Analysis	3
4.2. Feasibility Study	4
4.2.1. Technical Feasibility	4
4.2.2. Operational Feasibility.....	4
4.2.3. Economic Feasibility	5
4.2.4. Schedule Feasibility (Gantt Chart).....	6
4.3. High Level Design of System	6
4.3.1. System Architecture	6
4.3.2. System Working Mechanism of Proposed System:	6
4.3.3. Player Life and Health System	7
4.3.4. Difficulty Progression	7
5. Algorithms used	9
5.1. Finite State Machine (FSM) for Enemy AI.....	9
5.2. Difficulty Scaling Algorithm	10
5.3. Collision Detection Algorithm.....	10
6. Expected Outcome	11
References and Citations.....	12
APPENDICES	13
Appendix A: In-Game Screenshots.....	13
Appendix B: Key Source Code Snippets	15

B.1. Player's Movement	15
B.2. Enemy FSM Controller	15

LIST OF ABBREVIATIONS

Abbreviation	Full Form
2D	Two-Dimensional
AI	Artificial Intelligence
BIT	Bachelor of Information Technology
DDA	Dynamic Difficulty Adjustment
FSM	Finite State Machine
GDScript	Godot Script (Godot Engine's native scripting language)
HP	Health Points
HUD	Heads-Up Display
IEEE	Institute of Electrical and Electronics Engineers
RPG	Role-Playing Game
TU	Tribhuvan University
UI	User Interface

LIST OF FIGURES

Figure 1: Flowchart of Game Working Mechanism	8
Figure 2:Room design.....	13
Figure 3:Enemy death.....	13
Figure 4: Combat System.....	14
Figure 5: Key spawning after defeating enemies.....	14

LIST OF TABLES

Table 1:Functional requirements.....	3
Table 2:Non-Functional requirements	4
Table 3: Project schedules for 12 Weeks.....	6

1. Introduction

The video game industry has evolved significantly over the past few decades, becoming one of the most influential sectors in entertainment and interactive technology. Modern games are no longer limited to simple graphics and repetitive gameplay; instead, they integrate intelligent systems, dynamic environments, and interactive mechanics to create engaging user experiences. Among various game genres, Role-Playing Games (RPGs) are particularly popular because they combine exploration, combat, progression systems, and strategic decision-making. With the advancement of open-source game development platforms, it has become easier for students and independent developers to create sophisticated games using lightweight tools and efficient programming techniques.[1]

This project, titled *“Trapped Inside: 2D RPG Game”*, focuses on the development of a web-based 2D top-down RPG game using Godot Engine and GDScript. The game is designed around a room-based progression system where the player is trapped inside a sequence of rooms filled with enemies possessing different attack behaviors and movement patterns. To progress through the game, the player must defeat all enemies in a room, after which a key is spawned that unlocks the exit door leading to the next level. As the player advances, the difficulty gradually increases through stronger and faster enemies, creating a more challenging and engaging gameplay experience.

A major focus of this project is the implementation of Artificial Intelligence (AI) techniques for enemy behavior and game progression. The enemies are controlled using a Finite State Machine (FSM), allowing them to switch intelligently between multiple behavioral states such as Idle, Chase, Attack, and Dead. In addition, a dynamic difficulty scaling mechanism is implemented to enhance replayability and maintain player engagement by increasing enemy count, health, speed, and attack damage in each successive room. The project also emphasizes the use of open-source and cost-effective technologies for academic game development. Godot Engine provides an efficient environment for 2D rendering physics handling, animation, collision detection, and scene management without licensing costs. By utilizing Godot’s built-in systems such as CharacterBody2D, Area2D, TileMap, and AnimationPlayer, the game achieves responsive controls and smooth gameplay while remaining lightweight enough to run in modern web browsers and standard desktop systems.

2. Problem statement

Most beginner-level web games lack intelligent enemy behavior and meaningful progression systems. Enemies often follow predictable patterns, reducing replay value and player engagement. Additionally, many RPG games require heavy engines or installations, making them inaccessible on low-end systems.

The problem is to design a lightweight web-based RPG game that:

- i. Uses AI-driven enemies with varied behaviors
- ii. Provides progressive difficulty
- iii. Runs smoothly in a web browser
- iv. Offers engaging gameplay using simple yet effective algorithms

3. Objectives

The main objectives of this project are:

- i. To develop a 2D Web-Based RPG Game and implement AI-based enemy behavior and attack strategies
- ii. To create a room-based progression system with increasing difficulty and key-based escape mechanism

4. Methodology

4.1. Requirement Identification

4.1.1. Study of Existing System / Literature Review

A review of existing 2D RPG games, including browser-based dungeon crawlers and indie RPG titles, was conducted to identify common architectural patterns and design limitations. Many of such systems rely on static enemy movement routines, rudimentary collision detection schemes, and highly constrained AI logic that does not adapt to player behavior

Research papers and articles on game AI were reviewed, focusing on:

- Finite State Machines (FSM)
- Difficulty scaling techniques

4.1.2. Requirement Analysis

i. Functional Requirements:

Table 1:Functional requirements

S. N	Requirement	Description
1.	Player Movement	Player moves in 4 directions using WASD
2.	Player Attack	Players perform melee attacks to damage enemies in range.
3.	Enemy AI (FSM)	Each enemy operates under a 4-state FSM: Idle, Chase, Attack, Dead.
4.	Health System	Players start with 200 HP per life; HP decreases when attacked by enemies.
5.	Lives System	Players have 3 lives; losing all 3 triggers Game Over.
6.	Key Spawn	A key spawn only after all enemies in the current room are defeated.
7.	Room Transition	Collecting the key and entering the door loads the next room.
8.	Difficulty Scale	Each new room increases enemy count, HP, speed, and damage.
9.	Game Over Screen	Game Over screen displays when all 3 lives are depleted.

ii. Non-Functional Requirements:

Table 2:Non-Functional requirements

Category	Requirement
Performance	The game must run at a minimum of 30 FPS on standard desktop hardware.
Usability	Controls must be learnable within 60 seconds of first play; no tutorial required.
Portability	Must run on Windows, macOS, and Linux as a standalone executable without additional dependencies.
Maintainability	Code must be modular, use Godot signals for decoupling, and be commented for readability.
Reliability	The game must not crash during normal gameplay sequences.

4.2. Feasibility Study

4.2.1. Technical Feasibility

The project is technically feasible using:

- Godot Engine 4 for rendering and game management
- GDScript for game logic and AI scripting [2]
- Godot’s built-in TileMap, AnimationPlayer, and Physics Engine for core systems
- Godot’s built-in UI system (Control nodes) for menus and HUD
- Simple AI algorithms for enemy behavior

Godot Engine is open-source and free, making the system lightweight and efficient for academic development [3].

4.2.2. Operational Feasibility

The game is easy to operate:

- Simple keyboard controls

- No complex setup required – Godot provides an all-in-one editor
- Runs as a standalone executable on Windows, macOS, and Linux

4.2.3. Economic Feasibility

- Uses open-source tools
- No licensing costs
- Developed in standard hardware

Thus, it is economically feasible for academic implementation.

4.2.4. Schedule Feasibility (Gantt Chart)

Table 3: Project schedules for 12 Weeks

Phase / Activity	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12
Requirement Analysis												
Literature Review & Dataset Collection												
System Design												
Development & AI Integration												
Testing & Evaluation												
Documentation & Final Submission												

4.3. High Level Design of System

4.3.1. System Architecture

Trapped Inside uses Godot Engine 4's scene-based node architecture. The game is organized into independent scenes managed by a central GameManager autoload. The architecture has three primary layers:

- Input Layer: Captures keyboard events and routes them to the PlayerController.
- Logic Layer: Contains PlayerController, EnemyController (FSM), RoomManager, and DifficultyManager scripts.
- Rendering & Physics Layer: Managed by Godot's built-in 2D renderer (Sprite2D, AnimatedSprite2D, TileMap) and physics engine (CharacterBody2D, Area2D).

4.3.2. System Working Mechanism of Proposed System:

- The player interacts with the game environment and enemies within each room using keyboard and/or mouse controls.
- Enemy AI governs the movement and attack patterns of all enemies within a room.

- The game engine, powered by Godot, handles all rendering operations, collision detection between game entities, and state updates (such as health changes, death events, and room transitions) [3].
- Room progression is triggered when the player collects the key, which itself only spawns after the last enemy in the room has been defeated.

4.3.3. Player Life and Health System

- The player begins with 3 lives, each containing 200 Health Points (HP).
- HP decreases when the player is attacked by enemies. When HP reaches 0, the player loses one life and respawns in the same room with HP reset to 200.
- If the player completes a room without losing a life, HP is fully restored for the next room. If a life was lost, HP is restored to 200 but the lost life is not recovered.
- The game ends (Game Over) when all 3 lives are depleted.

4.3.4. Difficulty Progression

- Each new room escalates difficulty through an increase in the number of enemies, higher enemy health values, increased weapon damage, and faster enemy AI behavior.

Flowchart:

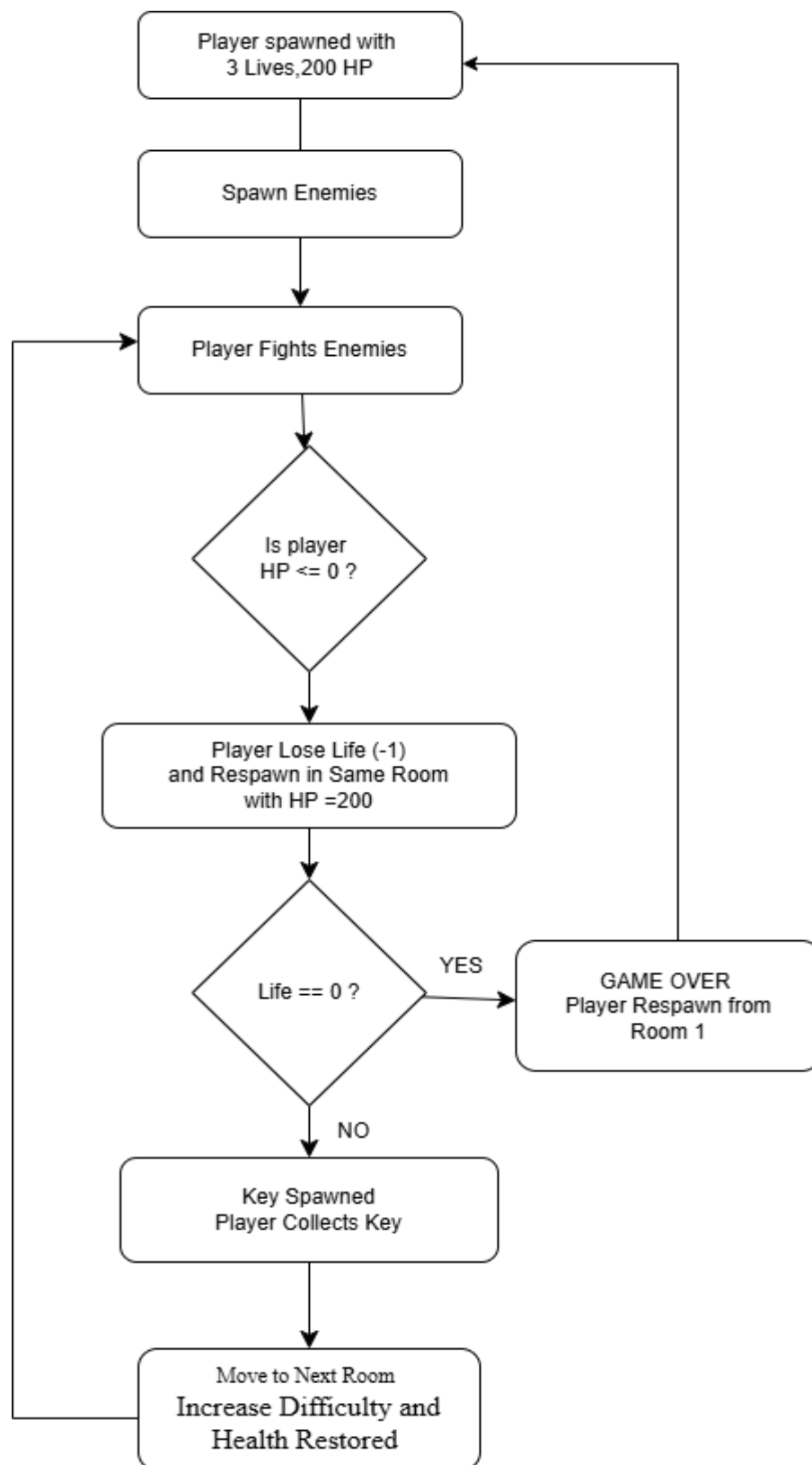


Figure 1: Flowchart of Game Working Mechanism

5. Algorithms used

5.1. Finite State Machine (FSM) for Enemy AI

Each enemy is governed by a Finite State Machine with four distinct states:

- Idle State: The enemy waits in place or patrols a predefined path, unaware of the player.
- Chase State: Triggered when the player enters the enemy's detection radius. The enemy actively follows and attempts to close distance with the player [4].
- Attack State: Activated when the player is within the enemy's attack range. The enemy performs its attack action.
- Dead State: The enemy has been defeated; it is removed from the game world and triggers a check for room completion.

Algorithm: Enemy AI using FSM

1. Start
2. Set enemy_state = IDLE
3. Store enemy_spawn_position
4. Loop while enemy is alive
 - If enemy health ≤ 0
 - enemy_state = DEAD
 - Else if enemy is currently attacking
 - enemy_state = ATTACK
 - Else if distance_to_player $\leq$ attack_range
 - enemy_state = ATTACK
 - Else if distance_to_player $\leq$ aggro_range
 - enemy_state = CHASE
 - Else if distance_from_spawn $>$ return_threshold
 - enemy_state = RETURN
 - Else
 - enemy_state = IDLE
5. Perform action based on enemy_state
 - If state = IDLE
 - Enemy stays near spawn location
 - If state = CHASE

```

    Enemy moves toward player
    If state = ATTACK
        Enemy performs attack animation
        Reduce player health
    If state = RETURN
        Enemy navigates back to spawn position
        When spawn reached → change state to IDLE
    If state = DEAD
        Remove enemy from scene
6. Repeat loop
7. End

```

5.2. Difficulty Scaling Algorithm

Room difficulty is dynamically scaled using the following parameters, which increase with each new room [5]:

- Enemy health is increased.
- The number of enemies spawned in each room is increased.
- Weapon damage output is increased, affecting both enemy and player weapons.
- Enemy AI behavior speed (movement speed, attack frequency) is increased.

5.3. Collision Detection Algorithm

- Detects player-enemy interactions
- Detects weapon hits
- Detects key pickup

Collision Detection Algorithm:

```

1. Start game loop
2. For each frame update
3. Detect collision between player attack hitbox and enemy

    If player attack hits enemy

        enemy.take_damage(player_damage)

    If enemy hitpoints <= 0

```

- trigger enemy death
- decrease enemy_count
- 4. Detect collision between enemy attack hitbox and player
 - if enemy attack hits player
 - player.take_damage(enemy_attack_damage)
 - if player health ≤ 0
 - reduce player life
 - respawn player
- 5. Detect collision between player and key object
 - if player collects key
 - unlock exit door
- 6. Detect collision between player and door
 - if door is unlocked and player enters door
 - load next room
- 7. Repeat until game ends
- 8. End

6. Expected Outcome

Upon successful completion of this project, the following outcomes are anticipated:

- A fully functional, web-based 2D RPG game that is accessible through any modern browser without installation.
- Multiple levels with clearly increasing difficulty, responsive animations, accurate collision detection, and reactive enemy AI behaviors.
- Sound effects and visual polish that enhance the overall player experience and game atmosphere.

References and Citations

- [1] G. N. Yannakakis and J. Togelius, *Artificial Intelligence and Games*, 2nd ed. Cham, Switzerland: Springer Nature, 2025
- [2] R. Orland, “GDScript vs. C#: Choosing a scripting language in Godot 4,” Godot Community, 2023. [Online]. Available: <https://godotengine.org/article/gdscript-progress-report-writing-a-new-parser/>. [Accessed: Feb. 2026].
- [3] Godot Engine Documentation, “Introduction to Godot 4,” Godot Engine, 2023. [Online]. Available: <https://docs.godotengine.org/en/stable/>. [Accessed: Feb. 2026].
- [4] C. W. Reynolds, “Steering behaviors for autonomous characters,” in Proc. Game Developers Conference (GDC), San Jose, CA, USA, 1999, pp. 763–782.
- [5] A. Zafar and U. Latif, “Dynamic difficulty adjustment in video games: A survey,” *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 13, no. 6, pp. 112–121, 2022.

APPENDICES

Appendix A: In-Game Screenshots



Figure 2:Room design

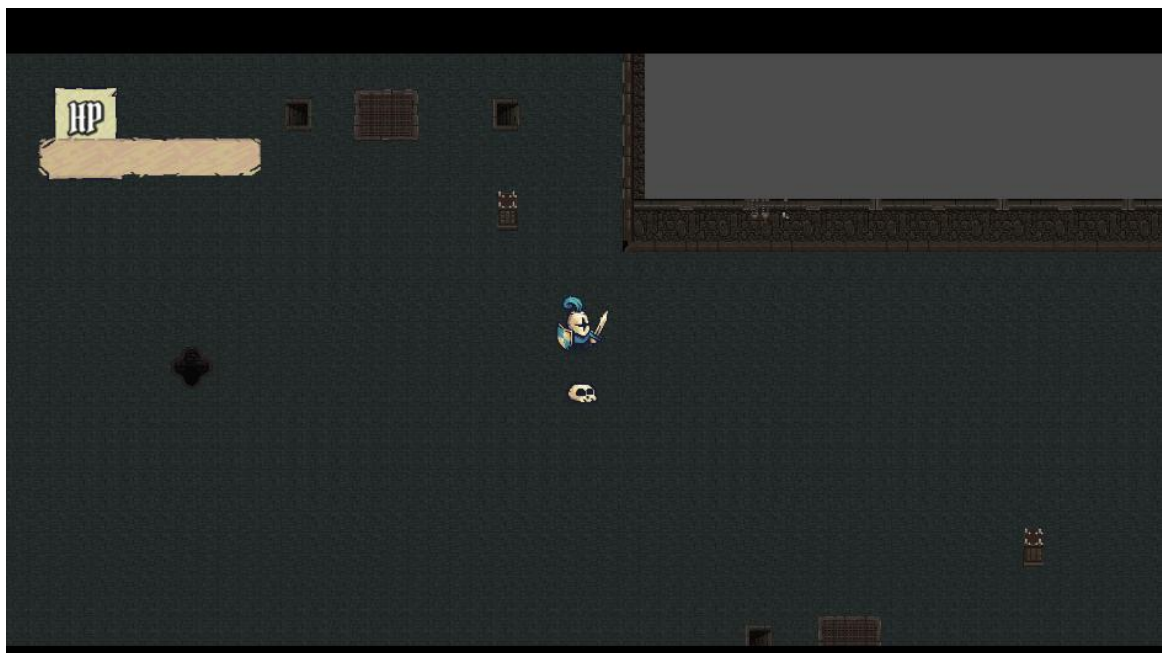


Figure 3:Enemy death



Figure 4: Combat System

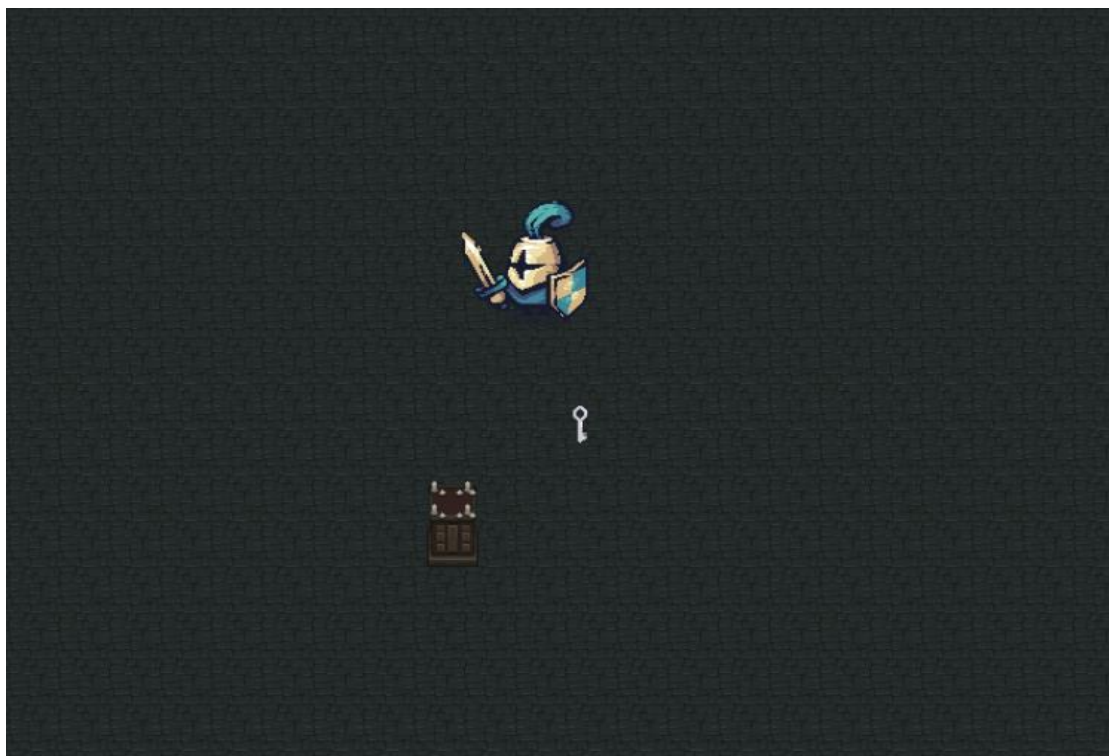


Figure 5: Key spawning after defeating enemies

Appendix B: Key Source Code Snippets

B.1. Player's Movement

```
func movement_loop():

    move_direction.x=int(Input.is_action_pressed("right"))-
    int(Input.is_action_pressed("left"))

    move_direction.y=int(Input.is_action_pressed("down"))-
    int(Input.is_action_pressed("up"))

    velocity=move_direction.normalized()*speed

    move_and_slide()

    if velocity!=Vector2.ZERO:state=State.RUN

    else:state=State.IDLE

    update_animation()

func attack():

    if state==State.ATTACK:return

    state=State.ATTACK

    update_animation()

    await get_tree().create_timer(attack_speed).timeout

    state=State.RUN

    if move_direction!=Vector2.ZERO

    else State.IDLE

    update_animation()
```

B.2. Enemy FSM Controller

```
func _physics_process(_delta):

    if state==State.DEAD:return

    if distance_to_player()<=attack_range:
```

```

        state=State.ATTACK

        attack()

    elif distance_to_player()<=aggro_range:

        state=State.CHASE

        move()

    else:

        state=State.IDLE

        velocity=Vector2.ZERO

    move_and_slide()

    update_animation()

func move():

    nav_agent.target_position=player.global_position

    velocity=global_position.direction_to(nav_agent.get_next_path_position())*speed

func attack():

    if is_attacking:return

    is_attacking=true

    await get_tree().create_timer(attack_speed).timeout

    is_attacking=false

```